

Pseudo Café

Restaurant Operations Platform

A production-grade, multi-tenant restaurant management system — built for real kitchens, real teams, and real revenue.

Framework	Next.js 15 · App Router
Database	Supabase · PostgreSQL
Auth	Supabase Auth + RBAC
Email	Resend SDK
Deployment	Vercel

PLATFORM OVERVIEW

What is Pseudo Café?

Pseudo Café is a full-stack, multi-tenant restaurant operations platform that manages the entire dine-in workflow — from a customer sitting down to a payment being received — in real time, across every role in a restaurant.

The problem it solves: Most restaurants still run on paper tickets, verbal communication, and separate billing apps. Pseudo Café replaces all of that with a unified digital system where the waiter, kitchen, and cashier all see the same live data instantly — on any device, with no page refresh.

■ Owner Dashboard

Live revenue counter, order count, average order value, table occupancy, and top-selling items — all updated in real time via WebSocket without page refresh.

■■ Waiter POS

Full-screen floor view with colour-coded table statuses. Search the menu, add items with per-item kitchen notes, and fire the order to the KDS in one tap.

■■■ Kitchen Display (KDS)

Live order board — no paper tickets. Items cycle Queued → Preparing → Ready with a single tap. Late orders (15 min+) highlighted in red with a ■ LATE badge.

■ Cashier Billing

Itemised bill, RBAC-capped percentage discount, Cash/Card/Other payment method, optional reference number. One click confirms payment and frees the table instantly.

■ Staff Management

Invite staff by email — credentials delivered via Resend. New staff show Pending until first login, then automatically flip to Active. Deactivate without deleting.

■ Sales Reports

Date-range reports (Today / 7 days / 30 days) — total revenue, breakdown by payment method, and top 10 items ranked by revenue.

ROLES & PERMISSIONS

Who uses what

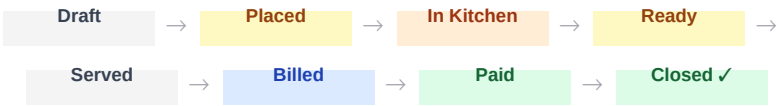
Every user is assigned a role at invite time. Permissions are enforced at three layers: the UI (buttons hidden), the API (server-side RBAC check), and the database (Row Level Security). A rogue API call cannot read another tenant's data.

Owner	Full access — dashboard, menu, staff, reports, all POS screens.
Manager	Same as Owner. Can invite staff, apply discounts, view reports.
Cashier	Cashier billing screen only. Discount capped by RBAC limit_value.
Waiter	Waiter POS only. Create orders, add notes, fire to kitchen, request bill.
Kitchen	Kitchen Display only. Advance item status. No access to prices or reports.

ORDER LIFECYCLE

From seat to receipt

Every order follows a strict server-side state machine. Transitions are validated at the API layer — an order cannot skip states, go backwards, or be paid before it is billed.



State	Who triggers	What happens in the system
draft	System (auto)	Created when waiter opens a table. Items can be added.
placed	Waiter	Order confirmed and locked. Items cannot be removed.
in_kitchen	Waiter	Order appears on KDS. Table card turns amber in real time.
ready	Kitchen (auto)	All items marked Ready → order auto-advances. Waiter sees update live.
served	Waiter	Food delivered to table. Request Bill button enabled.
billed	Waiter	Bill requested. Order appears instantly in Cashier queue.
paid → closed	Cashier	Payment recorded. Order closed. Table freed across all devices.

REAL-TIME ARCHITECTURE

What updates live — without refresh

Supabase Realtime uses PostgreSQL logical replication to stream row-level changes over WebSocket to every connected client. Three screens stay live simultaneously:

■ Floor View (Waiter)

Table cards update the instant another device changes table status. Two waiters cannot double-order the same table simultaneously.

■ Kitchen Display

New orders appear immediately — no polling, no refresh. The age timer ticks every 10 s. Items update as kitchen staff tap them on their screen.

■ Owner Dashboard

Revenue counter increments live when cashier processes a payment. Order count, top items, and table occupancy all update in real time via the same WebSocket channel.

■ Security layer

Supabase RLS policies ensure each WebSocket subscription only delivers rows belonging to the authenticated user's organisation — no cross-tenant data leakage.

HOW TO USE

Owner / Manager journey

1

Log in at /login

Enter email and password. Owner and Manager are redirected to /outlet/[id] — the management dashboard. The sidebar gives access to Overview, Menu, Staff, and Reports. A Sign out button sits at the bottom of the sidebar.

2

Build the menu — /outlet/[id]/menu

Click Add Item. Enter name, price, and category. Optionally paste an image URL — leave blank to use the auto-selected category photo (Unsplash). Toggle availability without deleting items for temporarily unavailable dishes (86'd).

3

Invite staff — /outlet/[id]/staff

Click Invite Staff. Enter full name, email, select a role, and set an initial password (minimum 8 characters). Staff receive a branded dark-theme welcome email showing their credentials. They appear with a Pending badge until their first login, then automatically become Active — no admin action required.

4

Monitor operations — Dashboard Overview

Live stat cards show today's revenue (increments on each cashier payment), order count, average order value, and table occupancy. Top 5 items sold appear in a bar chart below. Three quick-links open Waiter POS, Kitchen Display, and Cashier views directly.

5

Review performance — /outlet/[id]/reports

Switch between Today, Last 7 days, and Last 30 days. See total revenue, a breakdown by payment method (Cash / Card / Other), and a table of the top 10 items ranked by revenue with quantity sold.

HOW TO USE

Waiter journey

1

Open the floor — /waiter/[id]

Colour-coded table grid: Green = Free, Blue = Seated, Amber = Order in kitchen, Red = Bill requested. The grid updates live without refresh across all devices.

2

Tap a table → build order

The right panel opens with the full menu grouped by category. Use the search bar to find items instantly. Tap + to add, – to remove. Tap the note icon after adding an item to set special instructions (e.g. "no onions, extra sauce"). Notes travel to KDS.

3

Send to Kitchen

Cart total is shown at the bottom. Tap Send to Kitchen — the order fires to the KDS and the table card turns amber. If the API fails, an inline error banner shows the reason. You can add more items later; they join the existing order.

4

Request Bill

When food is served and the guest asks for the bill, tap Request Bill in the table header. The order moves to billed state and appears instantly in the Cashier queue. The table card turns red.

HOW TO USE

Kitchen Display & Cashier

- 1

Live order cards
Orders appear sorted oldest-first. Table label and elapsed time shown. Orders older than 15 minutes disappear and LATE badge.
- 2

Advance item status
Tap any item to cycle: Queued → Preparing → Ready. Kitchen notes from the waiter appear in message bubble at the bottom.
- 3

Auto-complete
When all items are Ready, the order auto-advances to ready state, disappears from the waiter's screen, and the table frees instantly across all devices.
- 1

Bill queue
Orders older than 15 minutes disappear and LATE badge. Click and drag to load the full bill.
- 2

Apply discount
Enter a percentage discount. The RBAC system silently caps it at the cash register's limit. The discount appears in message bubble at the bottom.
- 3

Confirm payment
Select Cash, Card, or Other. Opened floor view updates. Press OK to confirm. The order disappears from the waiter's screen and the table frees instantly across all devices.

STANDOUT FEATURES

What makes it extraordinary

■ Zero-polling real-time

Every screen uses Supabase Realtime (WebSocket). No refresh, no polling. Sub-second latency across all connected devices simultaneously — tested with KDS, floor view, and dashboard open at the same time.

■ Three-layer security

UI hides unauthorised buttons. API runs RBAC can() check server-side. Database enforces Row Level Security — even a rogue API call cannot read another tenant's data.

■ Multi-tenant by design

Every table has org_id. A single deployment hosts unlimited restaurant organisations. No data leakage at the DB layer. Ready to scale to SaaS with zero schema changes.

■ Branded staff onboarding

Welcome emails sent via Resend — dark-themed HTML with credentials card, role badge, and sign-in CTA. Staff show Pending until first login; auto-activate without any admin action.

■ Per-item kitchen notes

Waiters attach special instructions to individual items. Notes travel from POS → database → KDS in real time, eliminating verbal re-transmission and reducing kitchen errors.

■ State machine enforcement

Orders follow a strict state machine enforced at the API layer. Cannot skip states or pay before billing. Every transition is timestamped for auditability and revenue reconciliation.

TECH STACK

Built with

Next.js 15

App Router with React Server Components. Route groups for layout isolation. Server-side rendering for auth-protected pages.

Framework

TypeScript 6

Strict mode. All API responses, DB rows, and component props fully typed. Zero any escapes in business logic.

Language

Tailwind CSS v4

Utility-first CSS. Yellow/black glassmorphism design system. All styles inlined — no separate CSS files to manage.

Styling

Supabase

PostgreSQL database, Auth (email+password), Realtime subscriptions, Row Level Security. Managed cloud — free tier to Pro ready.

Backend

@supabase/ssr

Cookie-based auth for SSR. Session tokens in HTTP-only cookies. Middleware refreshes sessions on every request transparently.

Auth

Zod

Runtime schema validation on every API route. Malformed requests return structured 400 errors. Schemas shared client ↔ server.

Validation

Resend SDK

Transactional email — branded HTML welcome emails to invited staff. Reliable delivery. Free tier: 3,000 emails/month.

Email

TanStack Query v5

Server-state management with caching. Integrated with Supabase Realtime for cache invalidation on live events.

State

Lucide React

700+ MIT-licensed SVG icons used throughout all staff-facing UI — sidebar, stat cards, POS screens, and action buttons.

Icons

Framer Motion

Production-ready animations for the customer-facing pages — hero sections, card reveals, and page transitions.

Animation

Three.js + R3F

WebGL 3D for the customer-facing pizza customiser. React Three Fiber (R3F) and Drei helpers for declarative 3D scenes.

3D

Vercel

Zero-config Next.js deployment. Edge network, HTTPS, preview deploys on every PR, and env variable management.

Deploy

EXTERNAL SERVICES

What the platform depends on

Service	Purpose	Free Tier Limit	Upgrade When
Supabase supabase.com	PostgreSQL DB, Auth, Realtime, Storage, Row Level Security	500 MB DB · 50 K MAU · 200 concurrent connections	DB hits 500 MB or users exceed 50 K MAU
Resend resend.com	Transactional staff welcome emails with credentials	3,000 emails/month · 100 per day	Inviting more than 100 staff per day

Vercel vercel.com	Next.js hosting, CDN, serverless API functions	Hobby: 100 GB bandwidth · unlimited deploys	Commercial use or bandwidth > 100 GB/month
Unsplash CDN images.unsplash.com	Default food photos for menu item cards	Unlimited CDN reads — no API key used	Never — static CDN URLs, zero API calls

Total cost at demo/MVP scale: \$0/month. Every dependency is within its free tier for a single outlet with a normal team size. The platform is architected to upgrade to Supabase Pro and Vercel Pro on demand with no code changes.

DEPLOYMENT

From code to live URL

This guide takes you from a fresh clone to a publicly accessible deployment on Vercel, backed by Supabase cloud — in under 30 minutes. No DevOps, no servers to manage.

Phase 1 — External services setup

1 Create a Supabase project

Go to supabase.com → New project. Choose a region close to your users (ap-south-1 for Pakistan). After creation go to Project Settings → API and copy three values: Project URL → NEXT_PUBLIC_SUPABASE_URL, anon key → NEXT_PUBLIC_SUPABASE_ANON_KEY, service_role key → SUPABASE_SERVICE_ROLE_KEY.

2 Apply the database schema

In Supabase → SQL Editor, paste and run the full migration SQL. This creates all tables (organizations, outlets, users, roles, menu_items, orders, order_items, payments, tables, shifts), RLS policies, 13 performance indexes, and the role/permission seed data.

3 Seed demo data (optional)

Fill in your .env.local with the Supabase credentials, then run: npm run seed — this creates a demo org, outlet, 4 menu categories, 13 menu items with Unsplash images, 10 tables, and an owner account you can log in with immediately.

4 Create a Resend account

Go to resend.com → Sign up free → API Keys → Create key. Copy the re_... key as RESEND_API_KEY. On the free tier you can send from onboarding@resend.dev without any DNS setup. For production, add and verify your own domain under Resend → Domains.

Phase 2 — Local development

```
# 1. Clone and install
git clone https://github.com/your-org/pseudo-cafe.git
cd pseudo-cafe && npm install

# 2. Set up environment (fill in the 5 variables below)
cp .env.example .env.local

# 3. Start development server
npm run dev
# → http://localhost:3000
```

Environment variables

NEXT_PUBLIC_SUPABASE_URL	Your Supabase project URL — e.g. https://xyz.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public anon key from Supabase API settings (starts with sb_publishable_)
SUPABASE_SERVICE_ROLE_KEY	Secret service_role key — server-only, NEVER exposed to the browser
RESEND_API_KEY	API key from resend.com — starts with re_
NEXT_PUBLIC_SITE_URL	Your public URL — http://localhost:3000 in dev, https://yourapp.vercel.app in prod

Phase 3 — Deploy to Vercel

1 Push to GitHub

Create a new GitHub repository and push your code. Ensure `.env.local` is in `.gitignore` — never commit secrets.

```
git init && git add .
git commit -m "Initial commit"
git remote add origin https://github.com/your-org/pseudo-cafe.git
git push -u origin main
```

2 Import to Vercel

Go to vercel.com/new → Import Git Repository → select your repo. Framework preset is auto-detected as Next.js. Leave build settings as default.

3 Add environment variables

In the Vercel import screen → Environment Variables section, add all five variables. Set `NEXT_PUBLIC_SITE_URL` to your Vercel deployment URL (e.g. <https://pseudo-cafe.vercel.app>). You can update this after the first deploy.

4 Deploy

Click Deploy. Vercel builds the Next.js app (~90 seconds) and provisions serverless functions for all API routes. You get a live HTTPS URL with a global CDN instantly.

5 Update Supabase redirect URLs

In Supabase → Authentication → URL Configuration. Add your Vercel URL to Site URL and Redirect URLs. This is required for auth redirects to work in production.

6 Custom domain (optional)

Vercel → Project → Settings → Domains. Add your domain (e.g. app.pseudocafe.com). Vercel provides a CNAME record — add it to your DNS provider. SSL is auto-provisioned. Update `NEXT_PUBLIC_SITE_URL` and Supabase redirect URLs to the new domain.

Production readiness checklist

- | | |
|--|---|
| ✓ All 5 environment variables set in Vercel | ✓ <code>npm run build</code> passes with 0 TypeScript errors |
| ✓ Supabase Site URL updated to production domain | ✓ Supabase Realtime enabled on project |
| ✓ Database schema migration applied | ✓ Service role key is server-only (no <code>NEXT_PUBLIC_</code> prefix) |
| ✓ RLS policies enabled on all tables | ✓ <code>.env.local</code> is in <code>.gitignore</code> |
| ✓ Owner account created (seed or manually) | ✓ End-to-end order flow tested in staging |
| ✓ Resend domain verified for custom sender | ✓ Staff invite email delivery confirmed |
| ✓ Menu items added with images | ✓ KDS live updates verified on second device |
| ✓ Tables configured for the outlet | ✓ Supabase DB password is 20+ characters |

Full production stack: Custom domain (DNS) → Vercel Edge Network (CDN + SSL) → Next.js serverless functions (API routes) → Supabase PostgreSQL (database + auth + realtime) → Resend (transactional email). All connections encrypted in transit. No infrastructure to manage, patch, or scale manually.